

La Vogue Vista - Full Project Documentation

1. Project Summary

La Vogue Vista is a cross-platform mobile application built with Flutter that combines:

- E-commerce beauty product discovery and purchasing.
- Real-time AR try-on experiences (lip/makeup, hair color/style, nail).
- AI-assisted beauty analysis and recommendations.
- Firebase-backed authentication, data, and media infrastructure.

The application uses a hybrid architecture:

- Flutter/Dart for UI and product workflows.
- Native iOS/Android camera + rendering pipelines for live AR performance.
- On-device CV/ML where possible (MediaPipe + TensorFlow Lite).
- Cloud LLM integration for higher-level analysis/recommendation tasks.

2. Goals and Scope

Main goals

- Provide realistic live try-on for beauty/grooming products.
- Offer personalized guidance through AI analysis.
- Keep user experience responsive on mobile devices.
- Support end-to-end flow from discovery -> try-on -> purchase.

Current implemented scope

- User auth and profile bootstrapping.
- Firestore product catalog and shades.
- Cart and favorites.
- Live try-on modules for makeup and hair, plus nail try-on module (in active refinement).
- AI beauty assistant and hair suitability analysis.

3. Technology Stack

3.1 Frontend and state

- Flutter (Dart)
- Provider state management

3.2 Backend/platform services

- Firebase Core
- Firebase Auth
- Cloud Firestore
- Firebase Storage
- Firebase Analytics
- Firebase Crashlytics
- Firebase Messaging
- Google Sign-In

3.3 CV/ML and AI

- Google MediaPipe Tasks (face/hand/vision tasks)

- TensorFlow Lite (native + Flutter integrations)
- Gemini integration for assistant/analysis flows
- OpenRouter-based vision/LLM hair analysis pipeline

3.4 Native layers

- iOS: Swift + AVCapture + CoreAnimation + CocoaPods
- Android: Kotlin + Camera/native rendering path (project includes native platform view code)

4. Repository Structure (Key Areas)

- ``lib/`` - Flutter app features, services, providers, models, widgets.
- ``ios/`` - iOS native plugin, camera/render pipeline, Pod integration, run scripts.
- ``android/`` - Android app host and native integration.
- ``assets/`` - models, product images, UI assets.
- ``ios_quick_run.sh`` / ``ios_fast_dev.sh`` / ``ios_release_cycle.sh`` - execution helper scripts.

5. Application Architecture

5.1 High-level flow

1. App starts -> Firebase initialized.
2. Auth wrapper routes user to onboarding/login or dashboard.
3. Dashboard streams products from Firestore.
4. User can:
 - browse and purchase products,
 - open try-on modules,
 - use AI assistant/analysis screens.

5.2 Layering

- ****UI Layer****: Flutter screens and widgets.
- ****State Layer****: Providers (``auth``, ``cart``, ``makeup``).
- ****Data Layer****: Firebase services for auth/catalog/storage/cart/favorites.
- ****AR Layer****: Native platform view renderer via method/event channels.
- ****AI Layer****: MediaPipe/TFLite on-device + OpenRouter/Gemini for semantic recommendation tasks.

6. Core Flutter Modules

6.1 Auth and boot

- ``lib/main.dart``
- ``lib/providers/auth_provider.dart``
- ``lib/services/firebase_auth_service.dart``
- ``lib/firebase/firebase_config.dart``

Responsibilities:

- Firebase initialization.
- Session state management.
- Onboarding/login/signup routing.

6.2 Commerce/product modules

- ``lib/screens/dashboard_screen.dart``
- ``lib/screens/product_detail_page.dart``
- ``lib/screens/cart_screen.dart``
- ``lib/services/firestore_service.dart``
- ``lib/providers/cart_provider.dart``

Responsibilities:

- Product listing by category.
- Shade/product metadata retrieval.
- Add-to-cart and favorites.
- Purchase-oriented UI journey.

6.3 Try-on modules

- Live try-on: ``lib/screens/live_tryon_screen.dart``
- Full makeup workflow: ``lib/screens/full_makeup_screen.dart``
- Hair color: ``lib/screens/hair_color_tryon_screen.dart``
- Hair style match: ``lib/screens/hair_style_matcher_screen.dart``
- Nail flow: ``lib/screens/nail_tryon_landing.dart``, ``lib/screens/nail_tryon_screen.dart``
- Feature launcher: ``lib/screens/virtual_tryon_popup.dart``

6.4 AI modules

- Assistant UIs:
 - ``lib/screens/ai_beauty_assistant_screen.dart``
 - ``lib/screens/enhanced_ai_assistant_screen.dart``
- AI services:
 - ``lib/services/tflite_analysis_service.dart``
 - ``lib/services/gemini_chat_service.dart``
 - ``lib/services/openrouter_hair_analysis_service.dart``

Responsibilities:

- Facial/skin/hair analysis.
- Recommendation generation.
- Explainable suggestion narratives for user guidance.

7. Native iOS AR/ML Pipeline

Primary file: ``ios/Runner/NativeLipRendererPlugin.swift``

7.1 Bridge and rendering host

- Registers Flutter platform view and channels.
- Hosts camera session and CV pipeline in native Swift.
- Receives effect/category commands from Flutter (``cmd_*`` routing).

7.2 Camera and frame processing

- Uses ``AVCaptureSession`` with BGRA frames.
- Processes per-frame data via MediaPipe Tasks.
- Uses queue-based async processing for heavy segmentation tasks.

7.3 Overlay architecture

Multiple `CoreAnimation` layers are used:

- Lip overlay layer.
- Hair mask layer.
- Hair image/wig layer.
- Nail color/cuticle/highlight layers.
- Optional nail segmentation/debug layers.

7.4 Mode routing

By active category:

- Hair color mode: segmentation + recoloring mask pipeline.
- Hair style mode: frequent landmark tracking + throttled segmentation.
- Nail mode: landmark-driven geometry pipeline (hybrid path currently under active stabilization).
- Other makeup categories: face landmarks + shape overlays.

7.5 Performance techniques in code

- Frame discarding for stale frames.
- Background processing queues.
- Segmentation throttling per N frames.
- Temporal smoothing for geometry stability.
- Disabled implicit layer animations.
- Shared graphics context and memory-conscious processing.

8. AI and Analysis Logic

8.1 On-device inference

- MediaPipe facial and hand landmarks.
- TFLite-based segmentation/inference tasks.
- Used for real-time rendering and low-latency visual transformations.

8.2 Cloud-assisted reasoning

- OpenRouter/Gemini-powered semantic recommendation modules.
- Used for higher-level interpretation and explanation (e.g., hair suitability).
- Architecture intentionally separates visual inference (edge) from recommendation reasoning (cloud).

9. Data and Firebase Design

9.1 Firestore

- Product catalog and categories.
- Shade metadata per product.
- User cart/favorites/preferences.

9.2 Storage

- Product and feature media assets served through Firebase Storage where required.

9.3 Auth

- Email/password and Google sign-in flows.
- User profile initialization at signup/login.

10. Build, Run, and Dev Operations

10.1 Primary run scripts

- `ios\_quick\_run.sh` - standard debug run.
- `ios\_fast\_dev.sh` - attach-first fast loop.
- `ios\_release\_cycle.sh` - release build/install/launch helper.
- `ios/kill\_runner\_on\_device.sh` - stale process cleanup helper.

10.2 Typical local commands

- Flutter dependency sync: `flutter pub get`
- iOS pods sync: `cd ios && pod install`
- Quick run (wireless): `FLUTTER\_DEVICE\_CONNECTION=wireless ./ios\_quick\_run.sh`

11. Current Project Status

Stable/operational areas

- Auth and catalog flows.
- Firestore-integrated commerce journey.
- Live makeup and hair try-on architecture.
- AI assistant and analysis integration paths.

Active refinement areas

- Nail try-on robustness under varied finger poses and lighting/motion.
- Wireless iOS debug execution consistency.
- Native symbol/linkage cleanliness in iterative debug sessions.

12. Known Constraints and Risks

- Wireless iOS debugging introduces high startup latency and intermittent attach instability.
- Try-on quality depends on lighting, hand/face pose, camera noise, and model confidence.
- Some scripts include machine-specific defaults (device IDs/paths) and may need adaptation for other environments.
- Cloud recommendation features require valid API key configuration and network availability.

13. What Was Done (Implementation Highlights)

- Built integrated beauty platform with shopping + AR + AI in one app.
- Implemented native rendering bridge for real-time transformations.
- Added product seeding and Firestore catalog flow.
- Added multiple try-on experiences:
 - makeup/lip
 - hair color/style
 - nail try-on module
- Added AI recommendation workflows:
 - chat assistant
 - image-based hair/beauty analysis

- Added iOS run/iteration scripts to improve developer productivity.

14. Recommended Next Steps

1. Finalize nail tracking reliability with instrumentation-backed tuning.
2. Normalize category routing through enums/constants to reduce string mismatch risk.
3. Move all AI keys/secrets to secure runtime configuration.
4. Add feature-level performance telemetry (fps, segmentation cadence, frame drops).
5. Add regression test matrix for live try-on modes across representative devices.

15. Conclusion

La Vogue Vista demonstrates a practical hybrid mobile architecture that combines edge AI rendering, native camera processing, and cloud-assisted recommendation intelligence in a unified beauty platform. The project has achieved broad feature coverage across commerce, AR, and AI personalization, with nail try-on stability being the main remaining quality refinement area.